

Observation Task 5: Collection Framework – Method Implementation

(Part 1)

Observation Task: 05

Date:

02/10/2026

■ Objective / Aim

To design and implement a Java program that demonstrates the core methods of the Java Collection Framework interfaces – Collection, List, Set, Queue, Deque, Map, and Iterator – using standard implementing classes (ArrayList, HashSet, LinkedList, ArrayDeque, HashMap).

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

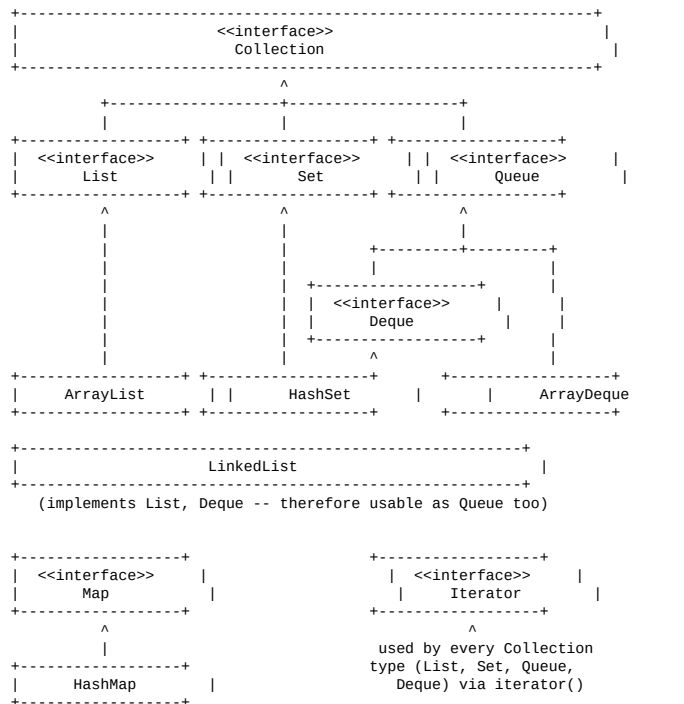
- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a class named **CollectionFrameworkDemo1**.
2. **List interface** – using ArrayList: demonstrate add(E e), add(int, E), get(int), set(int, E), remove(int), indexOf(), lastIndexOf(), subList(), and sort().
3. **Set interface** – using HashSet: demonstrate add(E e) (rejecting duplicates), remove(Object), contains(Object), size(), isEmpty(), and clear().
4. **Queue interface** – using LinkedList as a Queue: demonstrate add(), offer(), remove(), poll(), element(), and peek() showing FIFO order.
5. **Deque interface** – using ArrayDeque: demonstrate addFirst()/addLast(), offerFirst()/offerLast(), removeFirst()/removeLast(), pollFirst()/pollLast(), and peekFirst()/peekLast().
6. **Map interface** – using HashMap: demonstrate put(), get(), remove(), containsKey(), containsValue(), keySet(), values(), entrySet(), size(), isEmpty(), and clear().
7. **Iterator interface** – traverse a List using hasNext(), next(), and remove() to delete an element mid-iteration.
8. Print results after each operation to verify correct behavior for every method demonstrated.

Note: This Part 1 covers the core/base interfaces – Collection, List, Set, Queue, Deque, Map, Iterator. The ordered/navigable extensions – SortedSet, NavigableSet, SortedMap, NavigableMap, and ListIterator – are left for Part 2, since they build on the structures demonstrated here.

■ UML Class Diagram



Relationship:

- `List`, `Set`, and `Queue` are sub-interfaces of `Collection`.
- `Deque` extends `Queue` (every double-ended queue is also a queue).
- `ArrayList` implements `List`; `HashSet` implements `Set`.
- `LinkedList` implements **both** `List` and `Deque` (and therefore `Queue`), so it can be used as a list, a queue, or a stack.
- `ArrayDeque` implements `Deque` directly and is the more efficient choice when only queue/deque behavior is needed.
- `Map` is a **separate** root interface – it does **not** extend `Collection`, since it stores key-value pairs rather than single elements. `HashMap` implements `Map`.
- `Iterator` sits outside the `Collection/Map` hierarchy but is returned by every `Collection` type's `iterator()` method, giving a uniform way to traverse `List`, `Set`, `Queue`, and `Deque` alike.

■ Source Code

```

import java.util.*;

public class CollectionFrameworkDemo1 {

    public static void main(String[] args) {

        demoListInterface();
        demoSetInterface();
        demoQueueInterface();
        demoDequeInterface();
        demoMapInterface();
        demoIterator();
    }

    // ===== LIST =====
    static void demoListInterface() {
        System.out.println("==== LIST INTERFACE (ArrayList) =====");
        List<String> list = new ArrayList<>();

        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");
    }
}

```

```

list.add(1, "Mango"); // add(int index, E element)

System.out.println("After adds      : " + list);
System.out.println("get(2)         : " + list.get(2));

list.set(0, "Avocado"); // set(int index, E element)
System.out.println("After set(0,...) : " + list);

System.out.println("indexOf(Cherry) : " + list.indexOf("Cherry"));
System.out.println("lastIndexOf(Mango): " + list.lastIndexOf("Mango"));

List<String> sub = list.subList(1, 3); // subList(from, to)
System.out.println("subList(1,3)    : " + sub);

list.sort(Comparator.naturalOrder()); // sort(Comparator)
System.out.println("After sort()    : " + list);

list.remove(0); // remove(int index)
System.out.println("After remove(0) : " + list);
System.out.println();
}

// ===== SET =====
static void demoSetInterface() {
    System.out.println("==== SET INTERFACE (HashSet) =====");
    Set<Integer> set = new HashSet<>();

    set.add(10);
    set.add(20);
    set.add(10); // duplicate - rejected
    set.add(30);

    System.out.println("After adds (10 dup rejected): " + set);
    System.out.println("contains(20)      : " + set.contains(20));
    System.out.println("size()          : " + set.size());

    set.remove(20);
    System.out.println("After remove(20) : " + set);
    System.out.println("isEmpty()        : " + set.isEmpty());

    set.clear();
    System.out.println("After clear()    : " + set + " | isEmpty(): " + set.isEmpty());
    System.out.println();
}

// ===== QUEUE =====
static void demoQueueInterface() {
    System.out.println("==== QUEUE INTERFACE (LinkedList as Queue) =====");
    Queue<String> queue = new LinkedList<>();

    queue.add("Task1");
    queue.offer("Task2");
    queue.offer("Task3");

    System.out.println("Queue contents : " + queue);
    System.out.println("peek()        : " + queue.peek());
    System.out.println("element()     : " + queue.element());

    System.out.println("poll()        : " + queue.poll());
    System.out.println("After poll()  : " + queue);

    System.out.println("remove()      : " + queue.remove());
    System.out.println("After remove() : " + queue);
    System.out.println();
}

// ===== DEQUE =====
static void demoDequeInterface() {
    System.out.println("==== DEQUE INTERFACE (ArrayDeque) =====");
    Deque<Integer> deque = new ArrayDeque<>();

    deque.addFirst(2);
    deque.addLast(3);
    deque.offerFirst(1);
    deque.offerLast(4);

    System.out.println("Deque contents : " + deque);
    System.out.println("peekFirst()   : " + deque.peekFirst());
    System.out.println("peekLast()    : " + deque.peekLast());

    System.out.println("pollFirst()   : " + deque.pollFirst());

```

```

        System.out.println("pollLast()      : " + deque.pollLast());
        System.out.println("After polls     : " + deque);

        deque.removeFirst();
        System.out.println("After removeFirst(): " + deque);
        System.out.println();
    }

    // ===== MAP =====
    static void demoMapInterface() {
        System.out.println("===== MAP INTERFACE (HashMap) =====");
        Map<String, Integer> map = new HashMap<>();

        map.put("Ananya", 95);
        map.put("Rahul", 82);
        map.put("Priya", 88);

        System.out.println("Map contents      : " + map);
        System.out.println("get(Rahul)       : " + map.get("Rahul"));
        System.out.println("containsKey(Priya): " + map.containsKey("Priya"));
        System.out.println("containsValue(82) : " + map.containsValue(82));

        System.out.println("keySet()        : " + map.keySet());
        System.out.println("values()        : " + map.values());
        System.out.println("entrySet()      : " + map.entrySet());

        map.remove("Rahul");
        System.out.println("After remove(Rahul): " + map);
        System.out.println("size()          : " + map.size());
        System.out.println();
    }

    // ===== ITERATOR =====
    static void demoIterator() {
        System.out.println("===== ITERATOR INTERFACE =====");
        List<Integer> numbers = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5, 6));

        System.out.println("Before iteration : " + numbers);

        Iterator<Integer> it = numbers.iterator();
        while (it.hasNext()) {
            int value = it.next();
            if (value % 2 == 0) {
                it.remove(); // safely remove even numbers during iteration
            }
        }

        System.out.println("After removing evens via Iterator: " + numbers);
    }
}

```

■ Compilation & Execution Commands

Save the source code exactly as:

```
CollectionFrameworkDemo1.java
```

Compile the program:

```
javac CollectionFrameworkDemo1.java
```

Run the compiled program:

```
java CollectionFrameworkDemo1
```

■ Output Screen

```

===== LIST INTERFACE (ArrayList) =====
After adds      : [Apple, Mango, Banana, Cherry]
get(2)         : Banana
After set(0,...) : [Avocado, Mango, Banana, Cherry]
indexOf(Cherry) : 3
lastIndexOf(Mango): 1
subList(1,3)    : [Mango, Banana]
After sort()    : [Avocado, Banana, Cherry, Mango]
After remove(0) : [Banana, Cherry, Mango]

===== SET INTERFACE (HashSet) =====
After adds (10 dup rejected): [20, 10, 30]
contains(20)      : true

```

```

size()          : 3
After remove(20) : [10, 30]
isEmpty()       : false
After clear()   : [] | isEmpty(): true

===== QUEUE INTERFACE (LinkedList as Queue) =====
Queue contents  : [Task1, Task2, Task3]
peek()         : Task1
element()      : Task1
poll()         : Task1
After poll()   : [Task2, Task3]
remove()       : Task2
After remove() : [Task3]

===== DEQUE INTERFACE (ArrayDeque) =====
Deque contents  : [1, 2, 3, 4]
peekFirst()    : 1
peekLast()     : 4
pollFirst()    : 1
pollLast()     : 4
After polls    : [2, 3]
After removeFirst(): [3]

===== MAP INTERFACE (HashMap) =====
Map contents    : {Ananya=95, Rahul=82, Priya=88}
get(Rahul)     : 82
containsKey(Priya): true
containsValue(82) : true
keySet()       : [Ananya, Rahul, Priya]
values()       : [95, 82, 88]
entrySet()     : [Ananya=95, Rahul=82, Priya=88]
After remove(Rahul): {Ananya=95, Priya=88}
size()         : 2

===== ITERATOR INTERFACE =====
Before iteration : [1, 2, 3, 4, 5, 6]
After removing evens via Iterator: [1, 3, 5]

```

Note: *HashSet and HashMap do not guarantee insertion order, so the exact printed order of their elements may vary slightly across JVM runs.*

```

<terminated> CollectionFrameworkDemo1 [Java Application] C:\Users\togid\Downloa
===== SET INTERFACE (HashSet) =====
After adds (10 dup rejected): [20, 10, 30]
contains(20)      : true
size()           : 3
After remove(20)  : [10, 30]
isEmpty()        : false
After clear()     : [] | isEmpty(): true

===== QUEUE INTERFACE (LinkedList as Queue) =====
Queue contents    : [Task1, Task2, Task3]
peek()            : Task1
element()         : Task1
poll()            : Task1
After poll()      : [Task2, Task3]
remove()          : Task2
After remove()    : [Task3]

===== DEQUE INTERFACE (ArrayDeque) =====
Deque contents    : [1, 2, 3, 4]
peekFirst()      : 1
peekLast()       : 4
pollFirst()      : 1
pollLast()       : 4
After polls      : [2, 3]
After removeFirst(): [3]

===== MAP INTERFACE (HashMap) =====
Map contents      : {Rahul=82, Priya=88, Ananya=95}
get(Rahul)        : 82
containsKey(Priya): true
containsValue(82) : true
keySet()          : [Rahul, Priya, Ananya]
values()          : [82, 88, 95]
entrySet()        : [Rahul=82, Priya=88, Ananya=95]
After remove(Rahul): {Priya=88, Ananya=95}
size()            : 2

===== ITERATOR INTERFACE =====
Before iteration  : [1, 2, 3, 4, 5, 6]
After removing evens via Iterator: [1, 3, 5]

```

Fig: Console output of CollectionFrameworkDemo1 in Eclipse IDE

■ Result

The Java Collection Framework method-implementation program (Part 1) was successfully executed, demonstrating the core methods of the List, Set, Queue, Deque, Map, and Iterator interfaces through their standard implementing classes (ArrayList, HashSet, LinkedList, ArrayDeque, HashMap). The output confirms correct behavior for insertion, retrieval, removal, and traversal operations across each interface.